UNIT 1 - INTRODUCTION TO BIG DATA

Due to the massive digitalization, a large amount of data is being generated by web applications and Social networking sites that runs on internet by many organizations. In today's technological world the high computational power and large storage size is the basic need and it has been significantly increased over the period of time. The organizations are producing huge amount of data at rapid rate today and as per global internet usage report by Wikipedia, the 51% of the world's population uses internet to perform their day to day activities. Most of them use internet for web surfing, online shopping, or interacting using Social Medias sites like Facebook, twitter or LinkedIn etc. These websites generate massive amount of data that involve uploading and downloading of videos, pictures or text messages whose size is almost unpredictable with large number of users.

The recent survey on data generation says that Facebook produces 600 TB of data per day and analyzes 30+ Petabytes of user generated data, Boeing jet airplane generates more than 10 TBs of data per flight including geo maps and other information, Walmart handles more than 1 million customer transactions every hour with estimated more than 2.5 petabytes of data per day, there are 0.4 million tweets generated by twitter per minute, 400 hours of new videos uploads on YouTube with access by 4.1 million users. Therefore, it becomes necessary to manage such a huge amount of data generally called "Big data" in the perspective of its storage, processing and analytics.

In big data, the data generated in many formats like structured, semi structured or unstructured. The structured data has fixed pattern or schema which can be stored and managed using tables in RDBMS, The semi-structured data does not have pre-defined structure or pattern as it involves scientific or bibliographic data which can be represented using Graph data structures while unstructured data also do not have a standard structure, pattern or schema. The examples of unstructured data are videos, audios, images, pdfs, compressed, log or JSON files. The traditional database management techniques are incapable of storing, processing, handling and analyzing big data with various formats which includes images, audio, videos, maps, text, xml etc.

The processing of big data using traditional database management system is very difficult because of its four characteristics called 4 Vs of Big data shown in Fig. 1.1.1. In Big data, the Volume refers to size of data being generated per minute or seconds, Variety means types of data generated including structured, unstructured or semi structured

data, Velocity refers to speed of data generated per minute or per seconds and Veracity refers to uncertainty of data generated being generated.

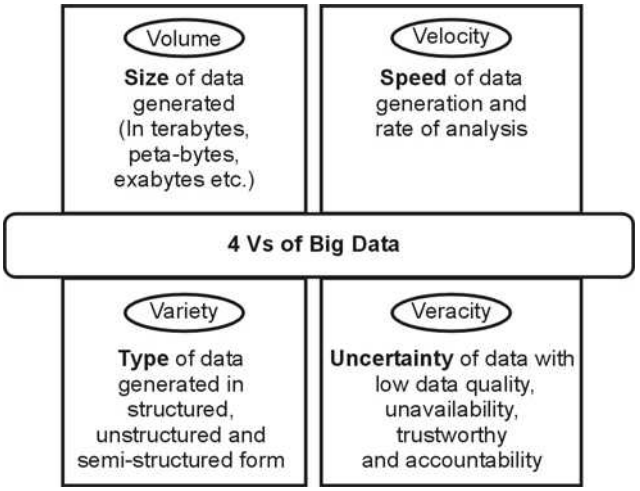

Fig. 1.1.1 : Four Vs of Big data

Because of above four V's, it becomes more and more difficult to capture, store, organize, process and analyze the data generated by various web applications or websites. In traditional analytics system, the cleansed or meaningful data is collected and stored by RDBMS in a data ware house. This data was analyzed by means of performing Extract, Transform and Load (ETL) operations. It has support of only cleansed structured data used for batch processing. The parallel processing of such data by traditional analytics were costlier because of expensive hardware. Therefore, big data analytics solutions came into picture which has many advantages over the traditional analytics solutions. The major advantages of Big data analytics are supporting real time or batch processing data, analyzes different formats of data, can process uncleansed or uncertain data, does not require an expensive hardware, supports huge volume of data generated at any velocity and perform data analytics at low cost.

Therefore, it is best to begin with a definition of big data. The analyst firm Gartner can be credited with the most-frequently used (and perhaps, somewhat abused) definition: Big data is high-volume, high-velocity and high-variety information assets that demand cost-effective, innovative forms of information processing for enhanced insight and decision making.

1.2 Evolution of Big Data

To deeply understand the consequences of Big Data analytics, the theory related to computing history, specifically Business Intelligence (BI) and scientific computing needs to be understood. The problem related to Big Data most likely be tracked before the evolution of computers, when unstructured data in paper format was needed to be tackled. Perhaps the first Big Data challenge came in to picture at Census Bureau, US, in 1880, where the information concerning approximately 60 million people had to be collected, classified, and reported that process took more than 10 years to process. Therefore, in 1890, the first Big Data platform was introduced with a mechanical device called the Hollerith Tabulating System which worked with punch cards with capacity of holding about 80 variables in one card which was very inefficient. So, in 1927, The Austrian-German engineer have developed a device that can store information magnetically on tape. But it also has very limited space for storage. In 1943, the British engineer had developed a machine called Colossus which was capable to scan 5.000 characters a second which reduced the workload from weeks to hours.

In 1969, the Advanced Research Projects Agency (ARPA), a subdivision of the Department of US Defense has developed ARPANET for military operations which evolves to the Internet in 1990. With the evolution of World wide web the true sense of big amount of data generation has started while after introduction of emerging technologies like internet of things. By 2013, the IoT had evolved to multiple technologies that uses Internet, wireless communications, embedded systems, Mobile technologies etc. As we know that the relational databases run on today's desktop computers have enough compute power to process the information contained in the 1890 census with some basic code. Therefore, the definition of Big Data continues to evolve with time and advances in technology.

1.3 Best Practices for Big Data Analytics

Like other technologies, there are some best practices that can be applied to the problems of Big Data. The best practices for Big data analytics are explained as follows.

1) **Start small with big data :** In Big data analytics, while analyzing the Big Data, always starts with a smaller task. Ideally, those smaller tasks will build the expertise needed to deal with the larger analytical problem. In Big data problem, the variety of data gets generated with uncover patterns and correlations in both structured and unstructured data. So, starting with a bigger task may create a dead spot in an analytics matrix where patterns may not be relevant to the problem being asked.

Therefore, every successful Big Data project always tend to start with smaller data sets and targeted goals.

2) **Think big for scalability :** While defining a Big data system, always follow a futuristic approach. That means determining how much data will be collected in next six months from now or calculating how many greater numbers of servers are needed to handle it. This approach will allow applications to be scaled easily without having any bottleneck.

3) **Avoid bad practices :** There are many potential reasons for failing Big Data projects. So, for making the successful Big data project, the following wrong practices must be avoided

   a) Rather than blindly adopting and deploying something, first understand the business purposes of the technology you are using for the deployment so as to implement the right analytics tools for the job at hand. Without a solid understanding of business requirements, the project will end up without having an intended outcome.

   b) Do not assume that the software will have all of the solutions for your problem as the business requirements, environment, input/output varies from project to project.

   c) Do not consider the solution of one problem relevant for every problem as each problem has unique requirements and needs a unique solution which can't be used to solve other problems. As a result, new methods and tools might be required to capture, cleanse, store, process at least some of your Big Data.

   d) Do not appoint same person for handling multiple types of analytical operations as lack of business requisite and analytical expertise may leads to failure of project. So, they require analytics professionals with statistical, actuarial, and other sophisticated skills, with expertise in advanced analytics operations.

4) **Treat big data problem as a scientific experiment :** In Big data project, collecting and analyzing the data is just a part of procedure while analytics is only producing the business value which needs to be incorporated into business processes intended to improve the performance and results. Therefore, every Big data problem requires a feedback loop for passing the success of actions taken as a result of analytical findings, followed by improvement of the analytical models based on the business results.

5) **Decide what data can be included and what to leave out :** As Big Data analytics projects involve large data sets that doesn't means all the data generated by a system can be analyzed. Therefore, it is required to select the appropriate datasets for analysis based on their value and outcomes.

6) **Must have a periodic maintenance plan :** The success of Big Data analytics initiative requires regular maintenance of analytics programs on the top of changes in business requirements.

7) **In-memory processing :** The In-memory processing of large datasets must be analyzed for getting the improvements in data-processing, speed of execution and volume of data. It gives hundreds of times of increased performance compared to older technologies, Better price-to-performance ratios, reductions in the cost of central processing units and memory and can handle rapidly expanding volumes of information.

1.4 Big Data Characteristics

Big data can be described by the following characteristics :

a) **Volume :** The quantity of data that is generated is very important in this context. It is the size of the data which determines the value and potential of the data under consideration and whether it can actually be considered Big Data or not. The name 'Big Data' itself contains a term which is related to size and hence the characteristic.

b) **Variety :** The next aspect of Big Data is its variety. This means that the category to which Big Data belongs to is also an essential fact that needs to be known by the data analysts. This helps the people, who are closely analyzing the data and are associated with it, to effectively use the data to their advantage and thus upholding the importance of the Big Data.

c) **Velocity :** The term 'velocity' in the context refers to the speed of generation of data or how fast the data is generated and processed to meet the demands and the challenges which lie ahead in the path of growth and development.

d) **Variability :** This is a factor which can be a problem for those who analyze the data. This refers to the inconsistency which can be shown by the data at times, thus hampering the process of being able to handle and manage the data effectively.

e) **Veracity :** The quality of the data being captured can vary greatly. Accuracy of analysis depends on the veracity of the source data.

f) **Complexity :** Data management can become a very complex process, especially when large volumes of data come from multiple sources. These data need to be linked, connected and correlated in order to be able to grasp the information that is supposed to be conveyed by these data. This situation, is therefore, termed as the 'complexity' of Big Data.

1.5 Validating the Promotion of the Value of Big Data

In previous sections, we have seen the characteristics and best practices for big data analytics. From that the key factors for successful big data technologies which are beneficial for organization are

- Reducing the capital and operational cost
- Does not needed high end servers as it can be run on commodity hardware
- Supports both Structured and unstructured data
- Supports high performance and scalable analytical operations
- Simple programming model for scalable applications

Previously, the implementation of high-performance computing systems was restricted to large organizations. But because of low budget many organizations were not able to implemented it. However, with the improvement market condition and economy, the high-performance computing systems has attracted many organizations who willing to invest in implementation of big data analytics. This is particularly valid for those associations whose financial limits were already too diminutive to even consider accommodating the venture.

There are many factors that needs to be considered before adapting any new technology like big data analytics. Any new technology can't be adapted blindly just because of its feasibility and popularity within the organization. So, without considering the risk factors the procured technology may fail and leading to the disappointment phase of the hype cycle which may nullify the expectations for clear business improvements. Therefore, before opting a new technology the five factors needs to be considered are sustainability of technology, feasibility, integrability, value and reasonability.

Apart from that the reality and hype about big data analytics must be checked before opting it. To review different between reality and hype, one must see what can be done with big data and what is said about that.

The Center for Economics and Business Research (CEBR) has published the advantages of big data as

- Provide improvements in the strategy, business planning, research and analytics leading to new innovation and the product development
- Optimized spending with improved customer marketing
- Provide predictive, descriptive and Prescriptive analytics for improving supply chain management
- Provide accuracy in fraud detection.

There are some more benefits promoted by inculcating business intelligence and data warehouse tools in big data like enhanced business planning with product analysis, optimized supply chain management with fraud detection and analysis of waste, and abuse of products.

1.6 Big Data Use Cases

The big data system is designed for providing high-performance capabilities over the elastically harnessed parallel computing resources with distributed storage. It is intended for providing optimized results over the scalable hardware and high-speed networks. The Apache Hadoop is the opensource framework for solving Big data problem. The typical Big Data Use Cases solved by Hadoop are given as follows

a) It provides support for Business intelligence by querying, reporting, searching, filtering, indexing, aggregating the datasets.

b) It provides tools for report generation, trend analysis, search optimization, and information retrieval.

c) It has improved performance for data management operations like log storage, data storage and archiving, followed by sorting, running joins, Extract, Transform and Loading (ETL) processing, data conversions, duplicate analysis and elimination.

d) It supports text processing, genome and protein sequencing, web crawling, workflow monitoring, image processing, structure prediction, and so on.

e) It also supports applications like data mining and analytical applications like facial recognition, social network analysis, profile matching, text analytics, machine learning, recommendation system analysis, web mining, information extraction, and behavior analysis.

f) It supports different types of analytics like predictive, prescriptive and descriptive along with functions like as document indexing, concept filtering, aggregation, transformation, semantic text analysis, pattern recognition, and searching.

1.7 Characteristics of Big Data Applications

Each Big data solution (like Hadoop) to is intended to solve a business problem in quicker time over the larger deployments subject to one or more of the following criteria:

1. Data throttling : The challenges related to existing solution running on traditional hardware due to throttling are data accessibility issue, data latency, limited data availability and limits on bandwidth.
2. Computation-restricted throttling : Due to the limitation of computing resources, the existing solution has Computation-restricted throttling where the expected computational performance has not been met with conventional systems.

3. Large data volumes : Due to the huge volume of data, the analytical application needs high rates of data creation and delivery.
4. Significant data variety : Due to the diversity of applications, the data generated by applications may have variety of data like structured or unstructured generated by different data sources.
5. Data parallelization : As big data application needs to process huge amount of data; the application's runtime can be improved through task or thread-level parallelization applied to independent data segments.

Some of the big data applications and their characteristics are given in Table 1.7.1.

Sr. No.	Application Name	Possible Characteristics
1	Fraud detection	Data throttling, Computation-restricted throttling, Large data volumes, Significant data variety, Data parallelization.
2	Data profiling	Large data volumes, Data parallelization
3	Clustering	Data throttling, Computation-restricted throttling, Large data volumes, Significant data variety, Data parallelization.
4	Price modelling	Data throttling, Computation-restricted throttling, Large data volumes, Data parallelization.
5	Recommendation System	Data throttling, Computation-restricted throttling, Large data volumes, Significant data variety, Data parallelization.

Table 1.7.1 : Characteristics of Big data applications

1.8 Perception and Quantification of Value

As we know that the three important facets of Big data system are organizational readiness, suitability of the business challenge and big data's contribution to the organization. Therefore, to test the Perception and Quantification of Value of Big data, the following criteria's must be tested.

a) Weather big data system is "Increasing the revenues of organization". This can be tested by using a recommendation engine.

b) Weather Big data system is "Lowering the costs" of organizations spending's like capital expenses (Capex) and Operational expenses (Opex)

c) Weather Big data system is "Increasing the productivity" by speeding up the process of execution with efficient results.

d) Weather Big data system is "Reducing the risk" while using big data platform collecting the data from streams of automated sensors and can provide full visibility.

1.9 Understanding Big Data Storage

Every big data application requires collection of storage and computing resources to achieve their performance and scalability within a runtime environment. The collection of four key computing resources essential for running a Big data application are

a) CPU or Processor : which allows multiple tasks to be executed simultaneously

b) Memory : which holds the data for faster processing in association with CPU

c) Storage : provides persistence storage of data.

d) Network : which provides the communication channel between different nodes through which the datasets are exchanged between processing nodes and storage nodes.

As single-node computers are incapable to process huge amount of data, that's why the high-performance platforms are used which composed of collections of computers with a pool of resources that can process massive amounts of data.

1.10 A General Overview Of High-Performance Architecture

The High-performance architecture of Big data is composed of connecting multiple nodes together through variety of network topologies. However, it discriminates the organization of computing and data across the network of storage nodes.

In this architecture, a master job manager is responsible for managing the pool of processing nodes along with assigns tasks and monitors the activity. While storage manager manages the data storage pool and distributes datasets across the collection of storage resources, which doesn't require any colocation of data and processing tasks. It is only intended for minimize the costs of data access latency.

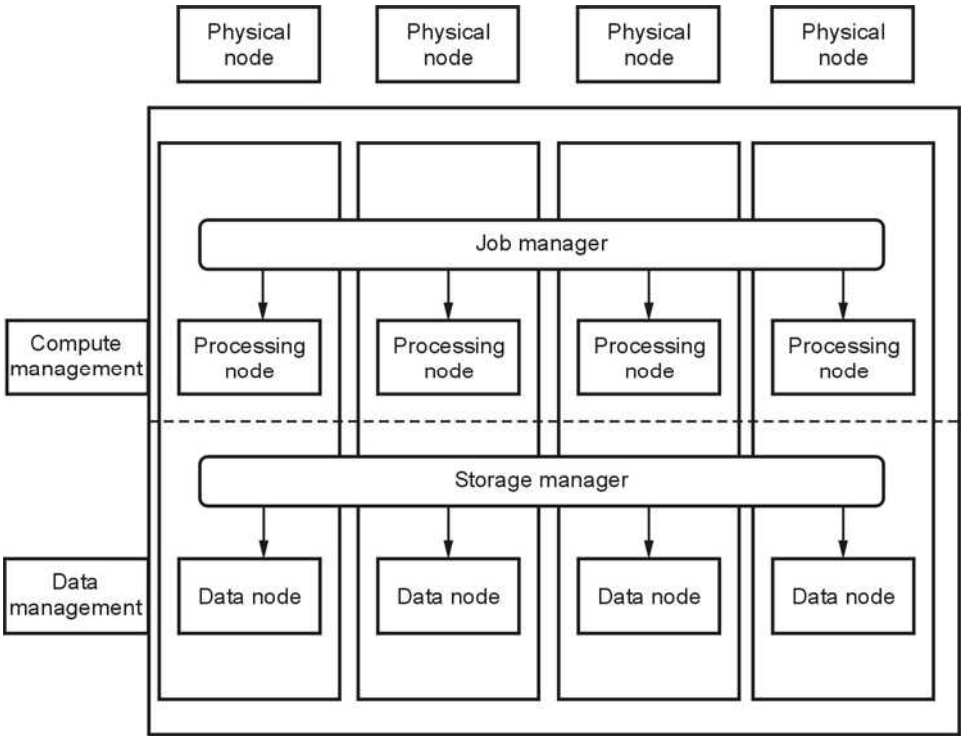

Fig. 1.10.1 : Generalize High-performance architecture of Big data System

To get a better understanding of the architecture for big data platform, we will examine the Apache Hadoop software stack, since it is a collection of open source projects that are combined to enable a software-based big data appliance. The generalized general overview of high-performance architecture of Hadoop is shown in Fig. 1.10.1.

1.11 Architecture of Hadoop

The challenges associated with Big data can be solved using one of the most popular frameworks provided by Apache is called Hadoop. Big data is a term that refers to data sets or combinations of data sets whose size (volume), complexity (variability), and rate of growth (velocity) make them difficult to be captured, managed, processed or analyzed by conventional technologies and tools, such as relational databases and desktop statistics or visualization packages, within the time necessary to make them useful.in simple way we can say Big data is a problem while hadoop is the solution for that problem.

The Apache Hadoop is an open source software project that enables distributed processing of large data sets across clusters of commodity servers using programming models. It is designed to scale up from a single server to thousands of machines, with a very high degree of fault tolerance. It is a software framework for storing the data and

running the applications on clusters of commodity hardware that provides massive storage for any kind of data, enormous processing power and the ability to handle virtually limitless concurrent tasks or jobs.

The Hadoop provides various tools for processing of big data collectively termed as Hadoop ecosystem. (See Fig. 1.11.1)

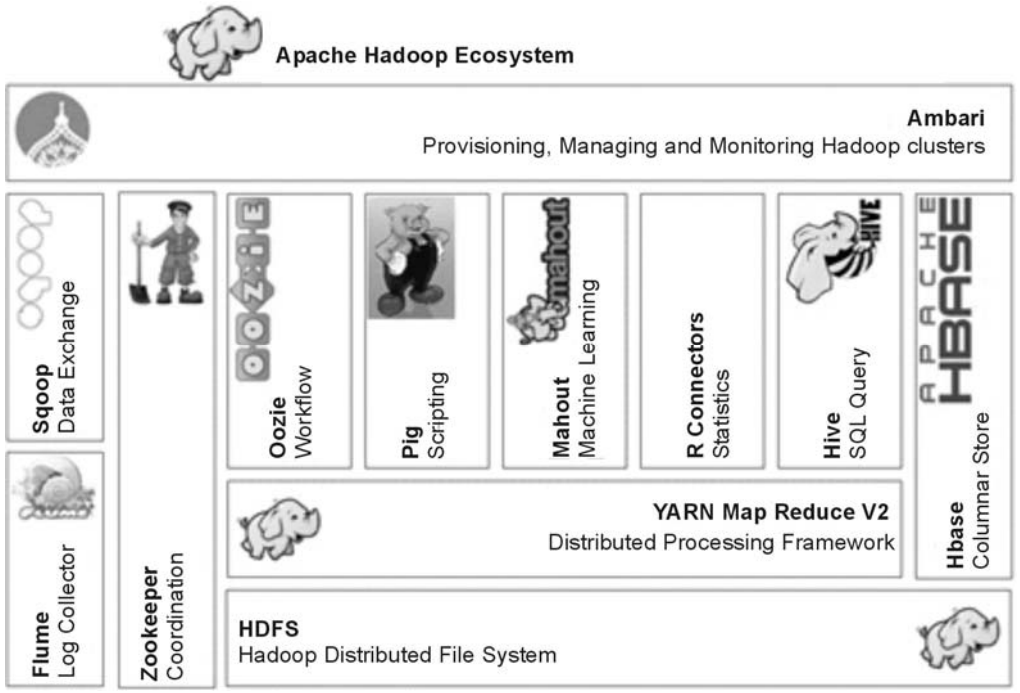

Fig. 1.11.1 Hadoop Ecosystem

The different components of Hadoop ecosystem are explained in following Table II.

Sr. No.	Name of Component	Description
1)	HDFS	It is a Hadoop distributed file system which is used to split the data in to blocks and is distributed among servers for processing. It runs multiple clusters to store several copies of data blocks which can be used in case of failure occurs.
2)	Map reduce	It's a programming method to process big data comprising of two programs written in Java such as mapper and reducer. The mapper extracts data from HDFS and put in to maps while reducer aggregate the results generated by mappers.
3)	Zookeeper	It is a centralized service used for maintaining configuration information with distributed synchronization and coordination.

4)	HBase	It is a Column-oriented database service used as NoSQL solution for big data
5)	Pig	It is a platform used for analyzing the large data sets using a high-level language. It uses dataflow language and provides parallel execution framework.
6)	Hive	It provides data warehouse infrastructure for big data
7)	Flume	It provides distributed and reliable service for efficiently collecting, aggregating, and moving large amounts of log data.
8)	Scoop	It is a tool designed for efficiently transferring bulk data between Hadoop and structured data stores such as relational databases.
9)	Mahaout	It provides libraries for scalable machine learning algorithms implemented on the top of Hadoop implemented using Map reduce framework.
10)	Oozie	It is a workflow scheduler system to manage the Hadoop jobs.
11)	Ambari	It provides a software framework for provisioning, managing and monitoring Hadoop clusters.

Table 1.11.1 : Different components of Hadoop ecosystem

1.12 Hadoop Distributed File System (HDFS)

The Hadoop Distributed File System (HDFS) is a hadoop implementation of distributed file system design that hold large amount of data and provide easier access to many clients distributed across the network. It is highly fault tolerant and designed to be run on low cost hardware (called commodity hardware).The files in HDFS are stored across the multiple machine in redundant fashion to recover the data loss in case of failure.

It enables storage and management of large files stored on distributed storage medium over the pool of data node. A Single name node runs in a cluster is associated with multiple data nodes that provide the management of hierarchical file organization and namespace. The HDFS file composed of fixed size blocks or chunks that are stored on data nodes. The name node is responsible for storing the metadata about each file that includes attributes of files like type of file, size, date and time of creation, properties of the files as well as the mapping of blocks to files at the data nodes. The data node treats each data block as a separate file and propagates the critical information with the name node.

The HDFS provides fault tolerance through data replication that can be specified at the time of file creation with attribute name degree of replication (i.e., the number of copies

made) which is progressively significant in bigger environments consisting of many racks of data servers. The significant benefits provided by HDFS are given as follows

- It provides Streaming access to file system data.
- It is suitable for distributed storage and processing.
- It is optimized to support high streaming read operations with limited set.
- It supports file operations like read, write, delete but append not update.
- It provides Java APIs and command line command line interfaces to interact with HDFS.
- It provides different File permissions and authentications for files on HDFS.
- It provides continuous monitoring of name nodes and data nodes based on continuous "heartbeat" communication between the data nodes to the name node.
- It provides Rebalancing of data nodes so as to equalize the load by migrating blocks of data from one data node to another.
- It uses checksums and digital signatures to manage the integrity of data stored in a file.
- It has built-in metadata replication so as to recover data during the failure or to protect against corruption.
- It also provides synchronous snapshots to facilitates rolled back during failure.

1.13 Architecture of HDFS

The HDFS follows Master-slave architecture using name and Data nodes. The Name node act as a master while multiple Data nodes worked as slaves. The HDFS is implemented as block structure file system where files are broken in to block of fixed size stored on hadoop clusters. The HDFS architecture is shown in Fig. 1.13.1.

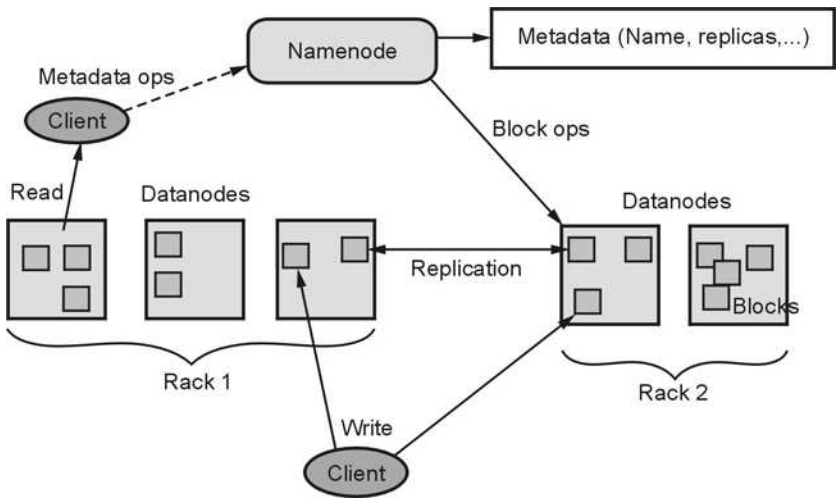

Fig. 1.13.1 : HDFS Architecture

The Components of HDFS composed of following elements

1) **Name Node :** An HDFS cluster consists of single name node called master server that manages the file system namespaces and regulate access to files by client. It runs on commodity hardware that manages file system namespaces. It stores all metadata for the file system across the clusters. The name node serves as single arbitrator and repository for HDFS metadata which is kept in main memory for faster random access. The entire file system name space is contained in a file called FsImage stored on name nodes file system, while the transaction log record are stored in Editlog file.

2) **Data Node :** In HDFS there are multiple data nodes exist that manages storages attached to the node that they run on. They are usually used to store users' data on HDFS clusters. Internally the file is splitted in to one or more blocks to data node. The data nodes are responsible for handling read/write request from clients. It also performs block creation, deletion and replication upon instruction from name node. The data node store each HDFS data block in separate file and several blocks are stored on different data nodes. The requirement of such a block structured file system is to store, manage and access files metadata reliably. The representation of name node and data node is shown in Fig. 1.13.2.

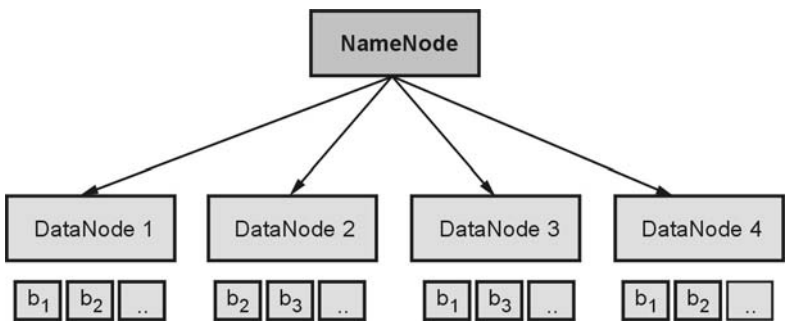

Fig. 1.13.2 : Representation of Name node and Data nodes

3) **HDFS Client :** In Hadoop distributed file system the user applications access the file system using the HDFS client. Like any other file systems, HDFS supports various operations to read, write and delete files, and operations to create and delete directories. The user references files and directories by paths in the namespace. The user application does not need to aware that file system metadata and storage are on different servers, or that blocks have multiple replicas. When an application reads a file, the HDFS client first asks the name node for the list of data nodes that host replicas of the blocks of the file. The client contacts a data node directly and requests the transfer of the desired block. When a client writes, it first asks the name node to choose data nodes to host replicas of the first block of the file. The client organizes a pipeline from node-to-node and sends the data. When the first block is filled, the

client requests new data nodes to be chosen to host replicas of the next block. The Choice of data nodes for each block is likely to be different.

4) **HDFS Blocks :** In general the users data stored in HDFS in terms of block. The files in file system are divided in to one or more segments called blocks. The default size of HDFS block is 64 MB that can be increase as per need.

The HDFS is fault tolerance such that if data node fails then current block write operation on data node is re-replicated to some other node. The block size, number of replicas and replication factors are specified in hadoop configuration file. The synchronization between name node and data node is done by heartbeats functions which are periodically generated by data node to name node.

Apart from above components the job tracker and task trackers are used when map reduce application runs over the HDFS. Hadoop Core consists of one master job tracker and several task trackers. The job tracker runs on name node like a master while task trackers runs on data nodes like slaves.

The job tracker is responsible for taking the requests from a client and assigning task trackers to it with tasks to be performed. The job tracker always tries to assign tasks to the task tracker on the data nodes where the data is locally present. If for some reason the node fails the job tracker assigns the task to another task tracker where the replica of the data exists since the data blocks are replicated across the data nodes. This ensures that the job does not fail even if a node fails within the cluster.

The HDFS can be manipulated either using command line. All the commands used for manipulating HDFS through command line interface begins with "hadoop fs" command. Most of the linux commands are supported over HDFS which starts with "-"sign.

For example: The command for listing the files in hadoop directory will be

```
#hadoop fs –ls
```

The general syntax of HDFS command line manipulation is

```
#hadoop fs -<command>
```

The most popular HDFS commands are given in Table 1.13.1

Sr. No.	Command	Description
1.	#hadoop fs –ls	List the files
2.	#hadoop fs –count hdfs:/	Count the number of directories, files and bytes under the paths
3.	#hadoop fs –mkdir /user/hadoop	Create a new directory hadoop under user directory

4.	#hadoop fs –rm hadoop/cust	Delete file cust from hadoop directory
5.	#hadoop fs –mv /user/training/custhadoop/	Move file cust from /user/training directory to hadoop directory
6.	#hadoop fs –cp/user/training/custhadoop/	Copy file cust from /user/training directory to hadoop directory
7.	#hadoopfs –copyToLocal hadoop/a.txt /home/training/	Copy file a.txt to local disk from HDFS
8.	#hadoopfs–copyFromLocal /home/training/a.txt hadoop/	Copy file a.txt from local directory /home/training to HDFS

Table 1.13.1 : HDFS Commands

1.14 Map Reduce and YARN

In Hadoop, MapReduce is the programming model for execution of Hadoop jobs which performs job management. The MapReduce execution condition utilizes an master/slave execution model, in which one master node is called the as JobTracker that deals with managing a pool of slave computing resources called TaskTrackers. The job of the JobTracker is to manage the TaskTrackers, continuously monitoring the accessibility, job management, scheduling the tasks, tracking the assigned tasks and ensuring the fault tolerance.

The job of the TaskTracker is a lot more straightforward : it wait for task assignment, execute the task and give status back to the JobTracker on periodic basis. The clients can make demands from the JobTracker, which turns into the sole arbitorator for allocation of resources. There are constraints inside this current MapReduce model. In the first place, the programming worldview is pleasantly fit to applications where there is region between the processing and the data, yet applications that request demand data movement will quickly move toward becoming impeded by latency issues. Second, not all applications are effectively mapped to the MapReduce model and **Third**, the designation of processing nodes within the cluster is fixed through allocation of certain nodes as “map slots” versus “reduce slots, When the computation is weighted toward one of the phases, the nodes assigned to the other phase are largely unused bringing about processor underutilization. This is being tended to in future renditions of Hadoop through the isolation of obligations inside a modification called YARN. In this approach, while in addition, there is the concept of an Application Master that is associated while taking over the responsibility for In this methodology, overall resource management has

been centralized and management of resources at each node is now performed by a local Node. Each application that directly negotiates with the central Resource Manager for resources there is the idea of an Application Master related with every application that legitimately consults with the Resource Manager for resource allocation, effective scheduling to improve node utilization and to provide monitoring progress with tracking status.

Last, the YARN approach enables applications to be better mindful of the data allocation over the topology of the resource inside a cluster. This mindfulness considers improved colocation of compute and data resources, reducing data movement, and thus, lessening delay related with data access latencies. The outcome ought to be expanded scalability and performance.

1.15 Map Reduce Programming Model

The map reduce is a programming model provided by Hadoop that allows expressing distributed computations on huge amount of data.It provides easy scaling of data processing over multiple computational nodes or clusters. In map reduces model the data processing primitives used are called mapper and reducer. Every map reduce program must have at least one mapper and reducer subroutines. The mapper has map method that transforms input key value pair in to any number of intermediate key value pairs while reducer has a reduce method that transform intermediate key value pairs that are aggregated in to any number of output key, value pairs.

The map reduce keeps all processing operations separate for parallel executions where a complex problem with extremely large in size is decomposed in to sub tasks. These subtasks are executed independently from each other. After that the result of all independent executions are combined together to get the complete output.

1.15.1 Features of Map Reduce

The different features provided by map reduce are explained as follows

- **Synchronization :** The map reduce supports execution of concurrent tasks. When the concurrent tasks are executed, they need synchronization. The synchronization is provided by reading the state of each map reduce operation during the execution and uses shared variables for those.
- **Data locality :** In map reduce, as the data resides on different clusters, it appears like a local to the users' application. To obtain the best result the code and data of application should reside on same machine.

- **Error handling :** Map reduce engine provides different fault tolerance mechanisms in case of failure. When the tasks are running on different cluster nodes during which if any failure occurs then map reduce engine find out those incomplete tasks and reschedule them for execution on different nodes.
- **Scheduling :** The map reduce involves map and reduce operations that divide large problems in to smaller chunks and those are run in parallel by different machines so there is a need to schedule different tasks on computational nodes on priority basis which is taken care by map reduce engine.

1.15.2 Working of Map Reduce Framework

The unit of work in map reduce is a job. During map phase the input data is divided in to input splits for analysis where each split is an independent task. These tasks run in parallel across hadoop clusters. The reducer phase uses result obtained from mapper as an input to generate the final result.

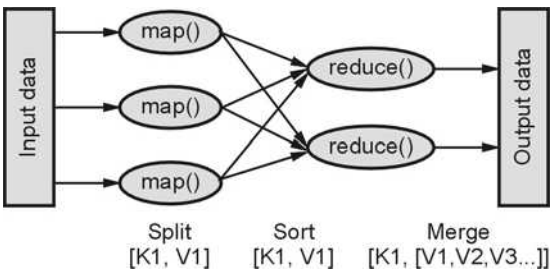

Fig. 1.15.1 : Map reduce process

The map reduce takes a set of input <key, value> pairs and produces a set of output <key, value> pairs by supplying data through map and reduce functions. The typical map reduce process is shown in Fig. 1.15.1.

Every map reduce program undergoes different phases of execution. Each phase has its own significance in map reduce framework. The different phases of execution in map reduce are shown in Fig. 1.15.2 (See on next page) and explained as follows

In input phase the large data set in the form of <key, value> pair is provided as a standard input for map reduce program. The input files used by map reduce are kept on HDFS (Hadoop Distributed File System) store which has standard Input Format specified by user.

Once input file is selected then the split phase reads the input data and divided those in to smaller chunks. The splitted chunks are then given to the mapper.

The map operations extract the relevant data and generate intermediate key value pairs. It reads input data from split using record reader and generates intermediate results. It is used to transform the input key, value list data to output key, value list which is then pass to combiner.

The combiner is used with both mapper and reducer to reduce the volume of data transfer. It is also known as semi reducer which accepts input from mapper and passes output key, value pair to reducer.

The shuffle and sort are the components of reducer. The shuffling is a process of partitioning and moving a mapped output to the reducer where intermediate keys are assigned to the reducer. Each partition is called subset and so each subset becomes input to the reducer. In general shuffle phase ensures that the partitioned splits reached at appropriate reducers where reducer uses http protocol to retrieve their own partition from mapper.

The sort phase is responsible for sorting the intermediate keys on single node automatically before they are presented to the reducer. The shuffle and sort phases occur simultaneously where mapped output is being fetched and merged.

The reducer reduces a set of intermediate values which share unique keys with set of values. The reducer uses sorted input to generate the final output. The final output is written using record writer by the reducer in to output file with standard output format.

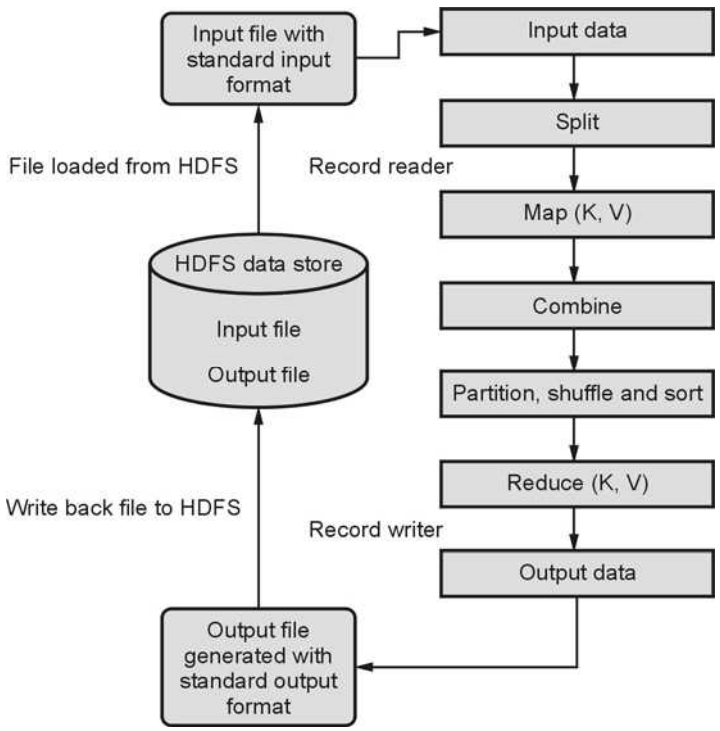

Fig. 1.15.2 Different phases of execution in map reduce

The final output of each map reduce program is generated with key value pairs written in output file which is written back to the HDFS store.

For example, of Word count process using map reduce with all phases of execution are illustrated in Fig. 1.15.3

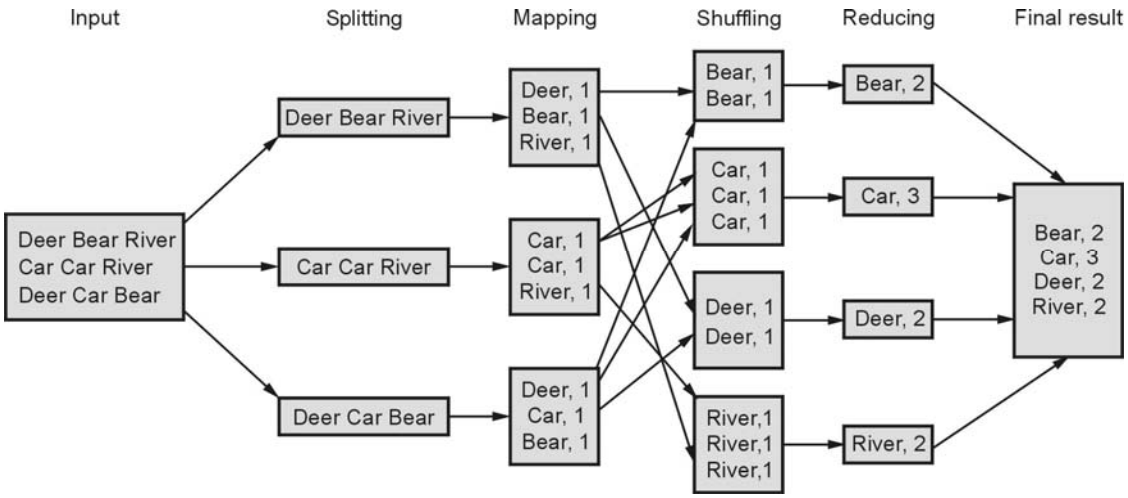

Fig. 1.15.3 : Word count process using map reduce

1.15.3 Input Splitting

The input to map reduce is provided by input file which has arbitrary format and resides on HDFS. The input and output format of files defines how the input files are splitted and how the output files are transformed. The common input output formats are TextInputFormat/ TextOutputFormat that reads or writes lines of text in to files, SequenceFileInputFormat/ SequenceFileOutputFormat that reads or write sequence of files those can be fed as a input to another map reduce jobs, KeyValueInputFormat that parses lines in to key, value pair and SequenceFileAsBinaryOutputFormat writes key, values to sequence file in binary format.

Once the input format is selected the next operation is to define the input splits that break the file in to tasks. The input split describes a unit of work which has at least a single map task. The map reduce functions are then applied to data sets collectively called job that has several tasks. The Input splits are often mapped with HDFS Blocks. The default size of HDFS block is 64 MB. The user can define the split value manually. Each split is processed by independent map function i.e. we can say that input split is the data processed by individual mapper. Basically, each split has number of records with key, value pair. The tasks are processed as per split size where largest one is processed first. The minimum split size is usually 1 byte. The user can define split size greater than the HDFS block size but it is not always preferred. The input split performed on Input file is shown in Fig. 1.15.4 where record reader is used to read the records from input split and to give them to the mapper.

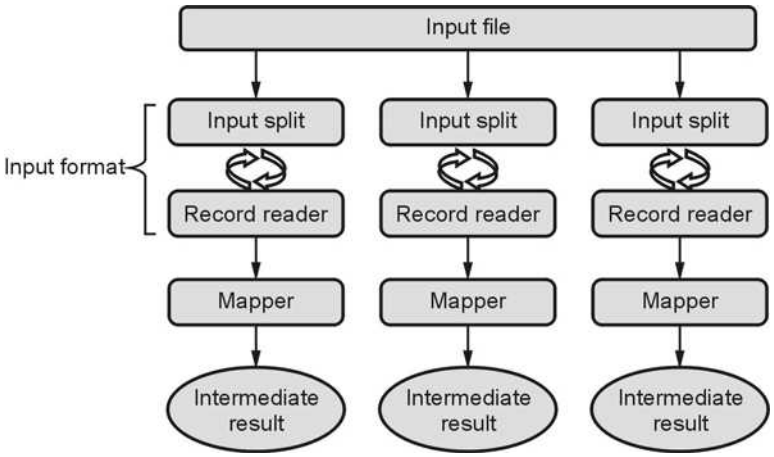

Fig. 1.15.4 : Input split performed on Input file

The split size can be calculated using Compute SplitSize () method by providing FileInputFormat. The input split is associated with record reader that loads data from source and convert it into key, value pair defined by Input Format.

1.15.4 Map and Reduce Functions

The map function process the data and generate intermediate key, value pair result while reduce function merges all intermediate values associated with particular key to generate output.

In map phase the list of data are provided one at a time to mapping function. It transfers each input element through map function to generate output data elements shown Fig. 1.15.5.

The reduce function receives an iterative input values from mappers output list which then aggregates the values together to return a single output. For reducer the input list is the mappers output list shown in Fig.1.15.6.

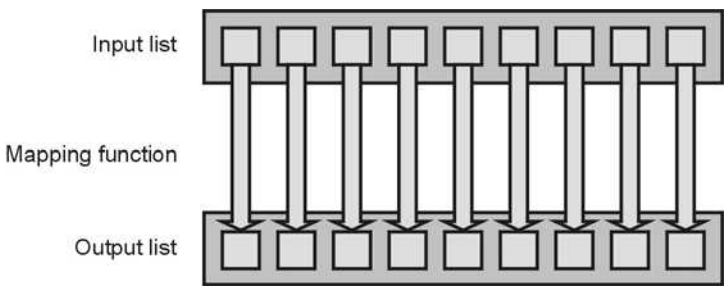

Fig. 1.15.5 : Mapper function

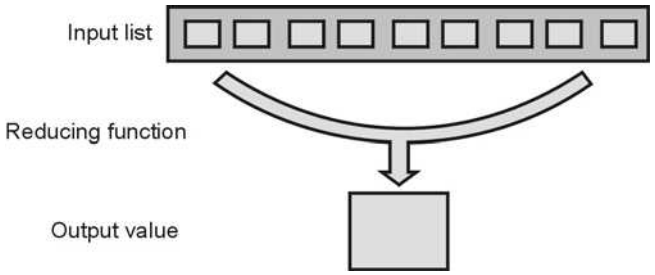

Fig. 1.15.6 : Reducer function

The input to mapper defined by input format while output generated by reducer generated in specified output format. if user does not specify input and output formats then Text format is selected by default.

1.15.5 Input and Output Parameters

As we have seen that the map reduce framework operates on key, value pairs. It accept input to the job as a set of key, value pair and produces set of key value pair as a output of the job shown in Fig.1.15.7.

Fig. 1.15.7 : Input output Key value pair

The input to mapper is defined by various input formats which validate the input jobs, split up the input files in to logical split and provide record readers for processing while output format validate the output of job and provide record writer to write output in to output file of the job.

The syntax of mapper and reducer classes in java are given below

Mapper Class	Reducer Class

public class MyMapper extends MapReduceBase implements Mapper<LongWritable, Text, Text, IntWritable>
{
public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException
{
……
……
}
}
	public class MyReducer extends MapReduceBase implements Reducer<text, IntWriable, Text, IntWritable>
	{
	public void reduce(Text key, Iterator<IntWritable > values, OutputCollector< Text, Inwritable > output, Reporter reporter) throws IOException
	{
	……
	……
	}
	}

The input to mapper is input key, input value and context which give information about current task while input to reducer is set of key values with output collector that collects key, value pair output from mapper or reducer. The reporter is used to report progress, update counters and status information of map reduce job.

Summary

- Due to the massive digitalization, a large amount of data is being generated by web applications and Social networking sites that runs on internet by many organizations such data is called Big data.
- In big data, the data is generated in many formats like structured, semi structured or unstructured.
- The structured data has fixed pattern or schema which can be stored and managed using tables in RDBMS, The semi-structured data does not have pre-defined structure or pattern as it involves scientific or bibliographic data which can be represented using Graph data structures while unstructured data also do not have a standard structure, pattern or schema.
- The processing of big data using traditional database management system is very difficult because of its four characteristics called 4 Vs of Big data those are Volume, Variety, Velocity and Veracity.
- The first Big Data challenge came in to picture at Census Bureau, US, in 1880, where the information concerning approximately 60 million people had to be collected, classified, and reported that process took more than 10 years to process.
- The Apache Hadoop is the open source framework for solving Big data problem.
- The common Big data use cases are Business intelligence by querying, reporting and searching the datasets, using big data analytics tools for report generation, trend analysis, search optimization, and information retrieval and performing predictive, prescriptive and descriptive analytics.
- The popular applications of Big data analytics are Fraud detection, Data profiling, Clustering, Price modelling and Recommendation System.

Two Marks Questions with Answers [Part A - Questions]

Q.1 Define Big data and also enlist the advantages of Big data analytics.

Ans. : Big data is high-volume, high-velocity and high-variety information assets that demand cost-effective, innovative forms of information processing for enhanced insight and decision making.

The major advantages of Big data analytics are :

- Supporting real time and batch data processing
- Supports huge volume of data generated at any velocity
- Reducing the capital and operational cost
- Does not needed high end servers as it can be run on commodity hardware
- Supports both Structured and unstructured data
- Supports high performance and scalable analytical operations
- Simple programming model for scalable applications
- It can process uncleansed or uncertain data

Q.2 What are the characteristics of Big data applications ?

Ans. : Big data can be described by the following characteristics :

1. **Volume -** The quantity of data that is generated is very important in this context. It is the size of the data which determines the value and potential of the data under consideration and whether it can actually be considered Big Data or not. The name 'Big Data' itself contains a term which is related to size and hence the characteristic.
2. **Variety -** The next aspect of Big Data is its variety. This means that the category to which Big Data belongs to is also an essential fact that needs to be known by the data analysts. This helps the people, who are closely analyzing the data and are associated with it, to effectively use the data to their advantage and thus upholding the importance of the Big Data.
3. **Velocity -** The term 'velocity' in the context refers to the speed of generation of data or how fast the data is generated and processed to meet the demands and the challenges which lie ahead in the path of growth and development.
4. **Variability -** This is a factor which can be a problem for those who analyze the data. This refers to the inconsistency which can be shown by the data at times, thus hampering the process of being able to handle and manage the data effectively.
5. **Veracity -** The quality of the data being captured can vary greatly. Accuracy of analysis depends on the veracity of the source data.
6. **Complexity -** Data management can become a very complex process, especially when large volumes of data come from multiple sources. These data need to be linked, connected and correlated in order to be able to grasp the information that is supposed to be conveyed by these data. This situation, is therefore, termed as the 'complexity' of Big Data.

Q.3 What are Features of HDFS ? **AU : May 17**

Ans. : The features of HDFS are given as follows :

- It is suitable for distributed storage and processing
- It provides Streaming access to file system data
- It is optimized to support high streaming read operations with limited set.
- It supports file operations like read, write, delete but append not update.
- It provides Java APIs and command line command line interfaces to interact with HDFS.
- It provides different File permissions and authentications for files on HDFS.
- It provides continuous monitoring of name nodes and data nodes based on continuous "heartbeat" communication between the data nodes to the name node.
- It provides Rebalancing of data nodes so as to equalize the load by migrating blocks of data from one data node to another.
- It uses checksums and digital signatures to manage the integrity of data stored in a file.
- It has built-in metadata replication so as to recover data during the failure or to protect against corruption.
- It also provides synchronous snapshots to facilitates rolled back during failure.

Q 4 What are the features provided by Map-Reduce programming model ? **AU : Nov.-18**

Ans. : The different features provided by map reduce are explained as follows :

- **Synchronization :** The map reduce supports execution of concurrent tasks. When the concurrent tasks are executed, they need synchronization. The synchronization is provided by reading the state of each map reduce operation during the execution and uses shared variables for those.
- **Data locality :** In map reduce, as the data resides on different clusters, it appears like a local to the users' application. To obtain the best result the code and data of application should reside on same machine.
- **Error handling :** Map reduce engine provides different fault tolerance mechanisms in case of failure. When the tasks are running on different cluster nodes during which if any failure occurs then map reduce engine find out those incomplete tasks and reschedule them for execution on different nodes.
- **Scheduling :** The map reduce involves map and reduce operations that divide large problems into smaller chunks and those are run in parallel by different machines. So there is a need to schedule different tasks on computational nodes on priority basis which is taken care by map reduce engine.

Q.5 What are different phases of executions in map-reduce programming ?

Ans. : Every map reduce program undergoes different phases of execution. Each phase has its own significance in map reduce framework. The different phases of execution in map reduce are shown in Fig. 1.1 and explained as follows :

In input phase the large data set in the form of <key, value> pair is provided as a standard input for map reduce program. The input files used by map reduce are kept on HDFS (Hadoop Distributed File System) store which has standard InputFormat specified by user. The map operations extract the relevant data and generate intermediate key value pairs. It reads input data from split using record reader and generates intermediate results. It is used to transform the input key, value list data to output key, value list which is then pass to combiner.

The combiner is used with both mapper and reducer to reduce the volume of data transfer. It is also known as semi reducer which accepts input from mapper and passes output key, value pair to reducer.

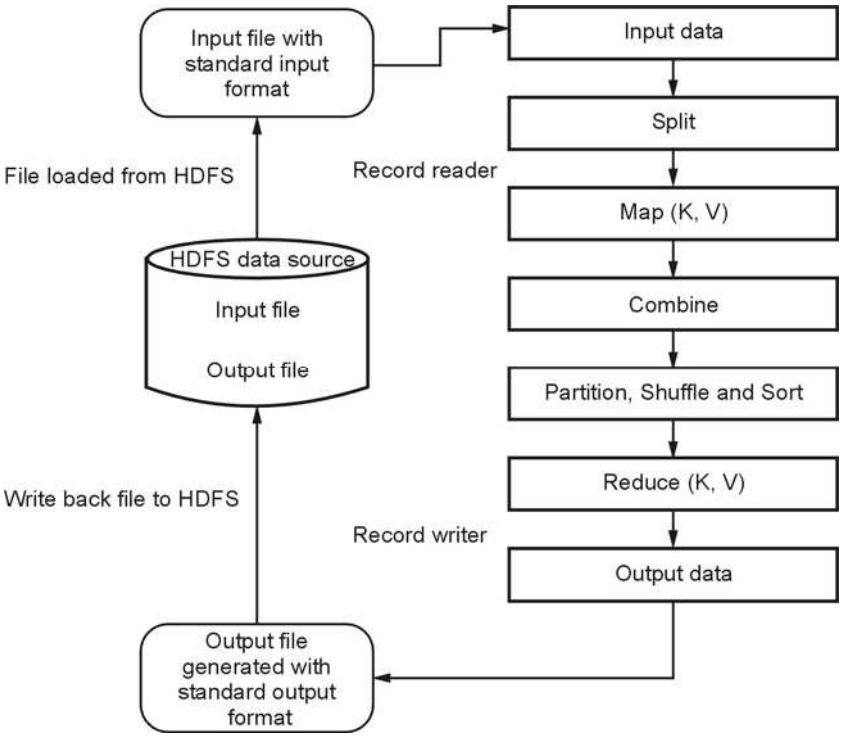

Fig. 1.1 : Different phases of execution in map reduce

The shuffle and sort are the components of reducer. The shuffling is a process of partitioning and moving a mapped output to the reducer where intermediate keys are assigned to the reducer. Each partition is called subset and so each subset becomes

input to the reducer. In general shuffle phase ensures that the partitioned splits reached at appropriate reducers where reducer uses http protocol to retrieve their own partition from mapper. The sort phase is responsible for sorting the intermediate keys on single node automatically before they are presented to the reducer. The shuffle and sort phases occur simultaneously where mapped output is being fetched and merged.

The reducer reduces a set of intermediate values which share unique keys with set of values. The reducer uses sorted input to generate the final output. The final output is written using record writer by the reducer in to output file with standard output format. The final output of each map reduce program is generated with key value pairs written in output file which is written back to the HDFS store.

Part - B Questions

Q.1 Explain Big data characteristics along with their use cases. **AU : May-17**

Q.2 What are 4 V's of Big data ? Also explain best practices for Big data analytics

Q.3 Explain generalized architecture of ?

Q.4 Explain architecture of Hadoop along with components of High-performance architecture of Big data.

Q.5 Explain the functionality of map-Reduce Programming model. **AU : May-17**

Q.6 Explain the functionality of HDFS and map-reduce in detail. **AU : Nov.-18**